

Computergrafik

Vorlesung 4

Hochschule Zittau/Görlitz

Christopher-Manuel Hilgner

Thema: Schatten & Reflexionen



Schatten

Schatten

Motivation

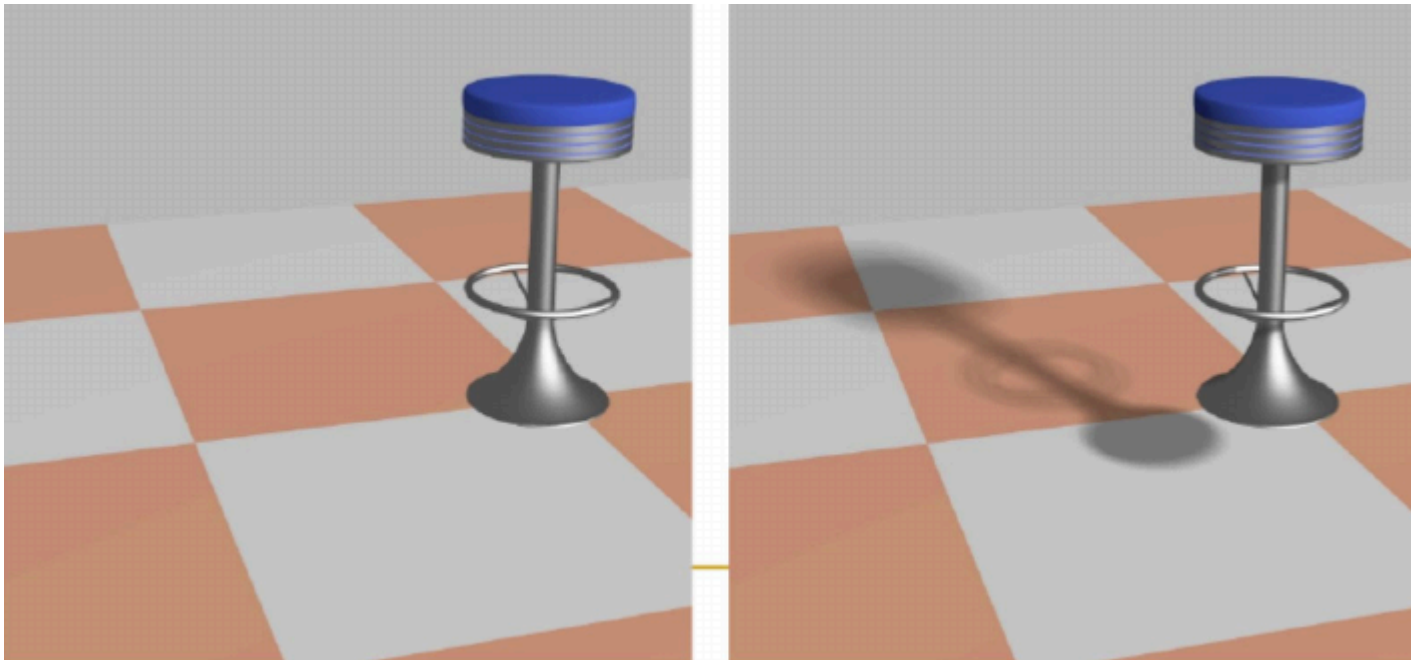
Motivation

Mehr Realismus und Atmosphäre



Motivation

- Hinweise über Rauminformationen
- Hilfestellung für Game Play und Orientierung



Motivation

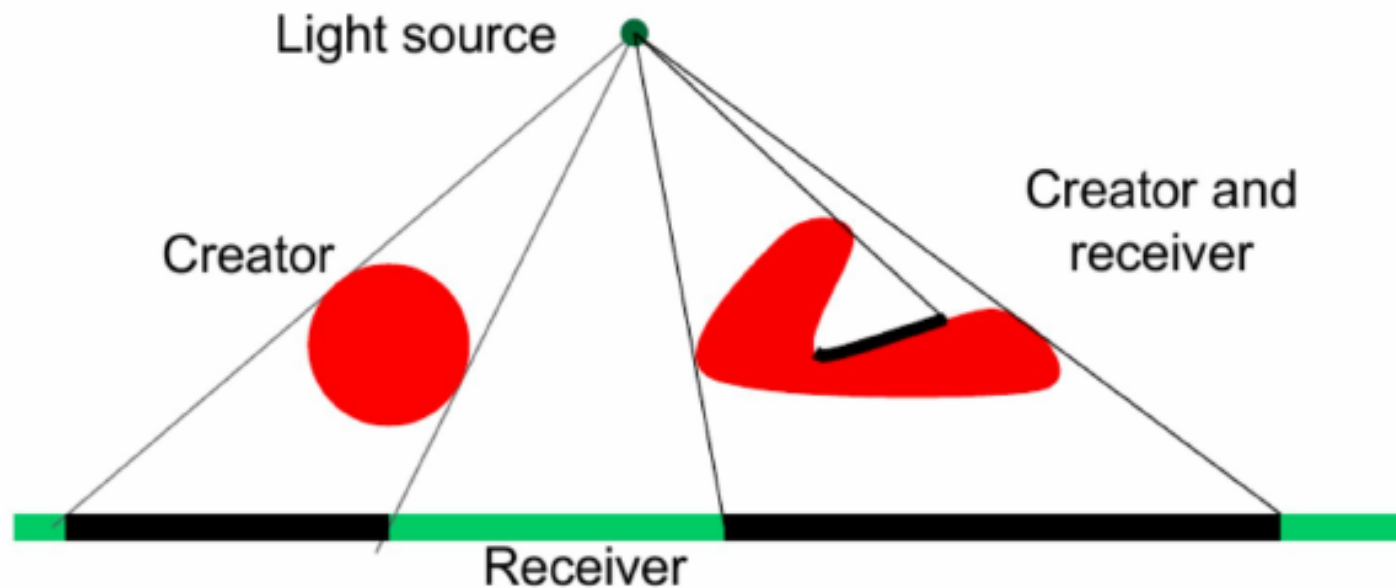
- Simple Schatten besser als keine Schatten
- das menschliche Auge *"vergibt"*



Definition

Bestandteile

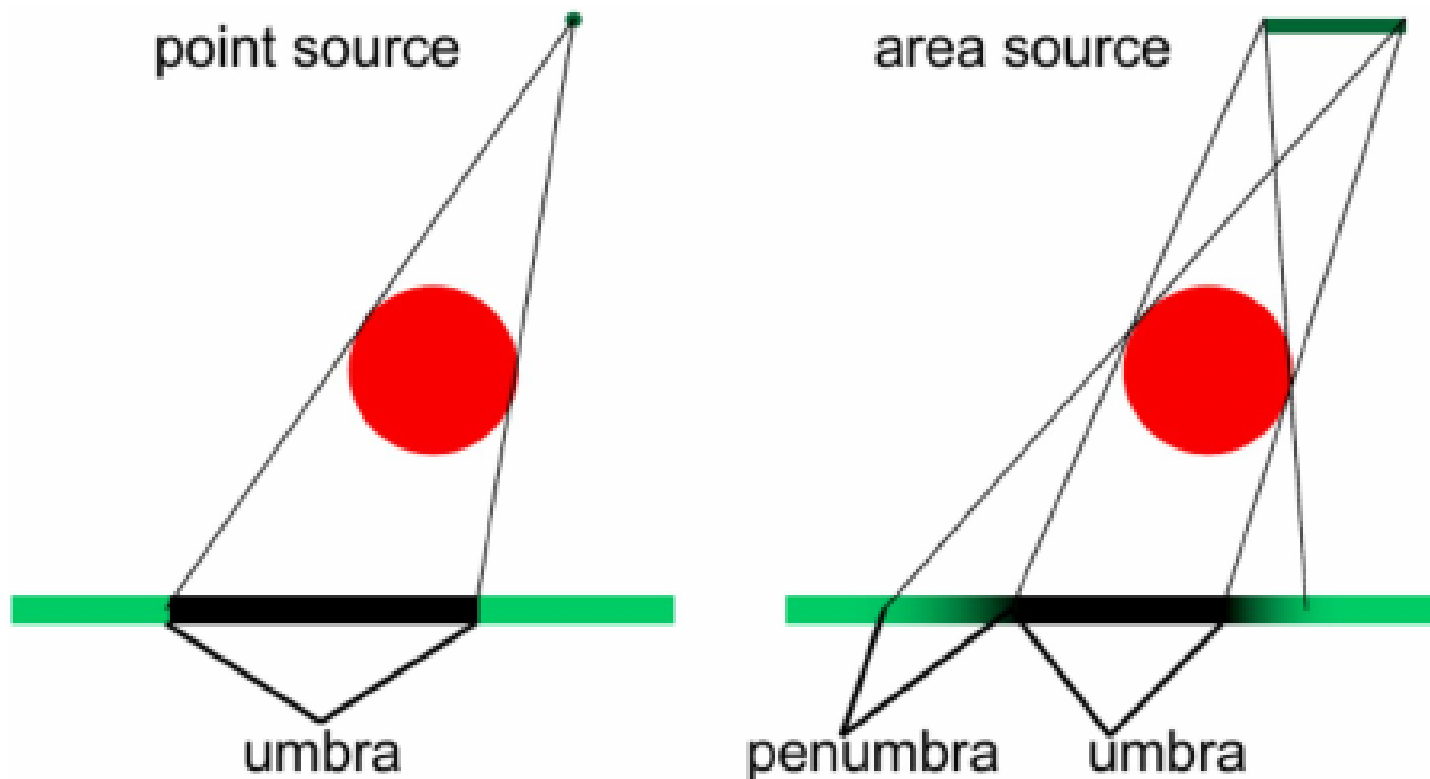
- Lichtquelle
- Schatten Creator und Receiver



Definition

Schattenarten

- Lichtquellentypen haben Einfluss auf Schatten
- Kernschatten (Umbra) und Halbschatten (Penumbra)



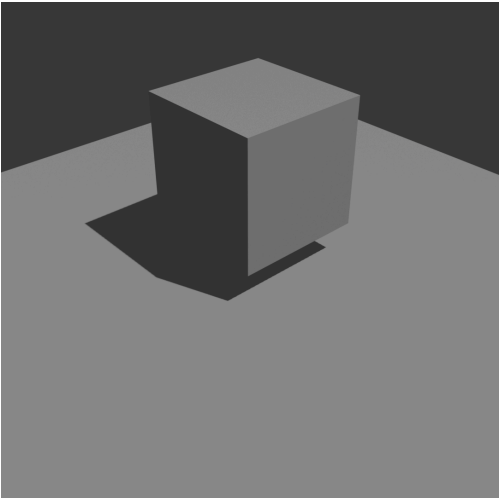
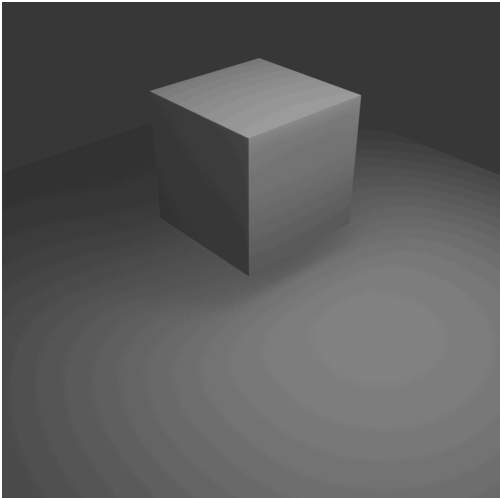
Definition

Schattenarten

- **Umbra:** Voller Schatten
- **Penumbra:** Halbschatten in einem teilweise schattiertem Bereich
- **Antumbra:** Halbschatten, wenn unterschiedliche Penumbra sich treffen
- **Hard Shadow:** Schatten (Umbra) mit harten Kanten
- **Soft Shadow:** Schatten mit weichen Kanten

Definition

Lichtquellen & Schattenarten

Hard Shadows	Soft Shadows
 <i>Hard Shadows</i> von einer Sonne als Punktlichtquelle	 <i>Soft Shadows</i> von einem Area Light

Definition

Denkweise über Schatten

- als separate Objekte
- als Volumina des Raums, die dunkel sind
- als Orte, die nicht von einer Lichtquelle aus gesehen werden, die auf die Szene blickt

Eigenschaften

- wenn die Lichtquellen Punktlichter sind, dann gibt es keinen Halbschatten und die Schatten haben harte Kanten
- Schatten sind ansichtsunabhängig
- Sie ändern sich in einer statischen Szene nicht, wenn die Ansicht geändert wird

Schatten

Modelle

Modelle

Simple Shadows

- sehr einfache Methode (Blinn, 1988)
 - beinhaltet das Zeichnen einer Projektion auf eine Grundplatte
 - begrenzt auf Ein-Objekte-Szenen
 - oder Szenen, wo Objekte keine Schatten auf sich gegenseitig werfen
- kann erweitert werden zu vertikalen Flächen
- kann erweitert werden zu zufällig orientierten Flächen



Shadow Mapping

Shadow Volumes

Shadow Maps

Modellierung

Shadow Mapping

- Eingeführt von Lance Williams 1978
- Überprüfung, ob ein Pixel von einer Lichtquelle sichtbar ist

Vorteile

- Kosten weniger empfindlich gegenüber geometrischer Komplexität
- Können an beliebiger Stelle abgefragt werden
- Oft einfacher zu implementieren

Nachteile

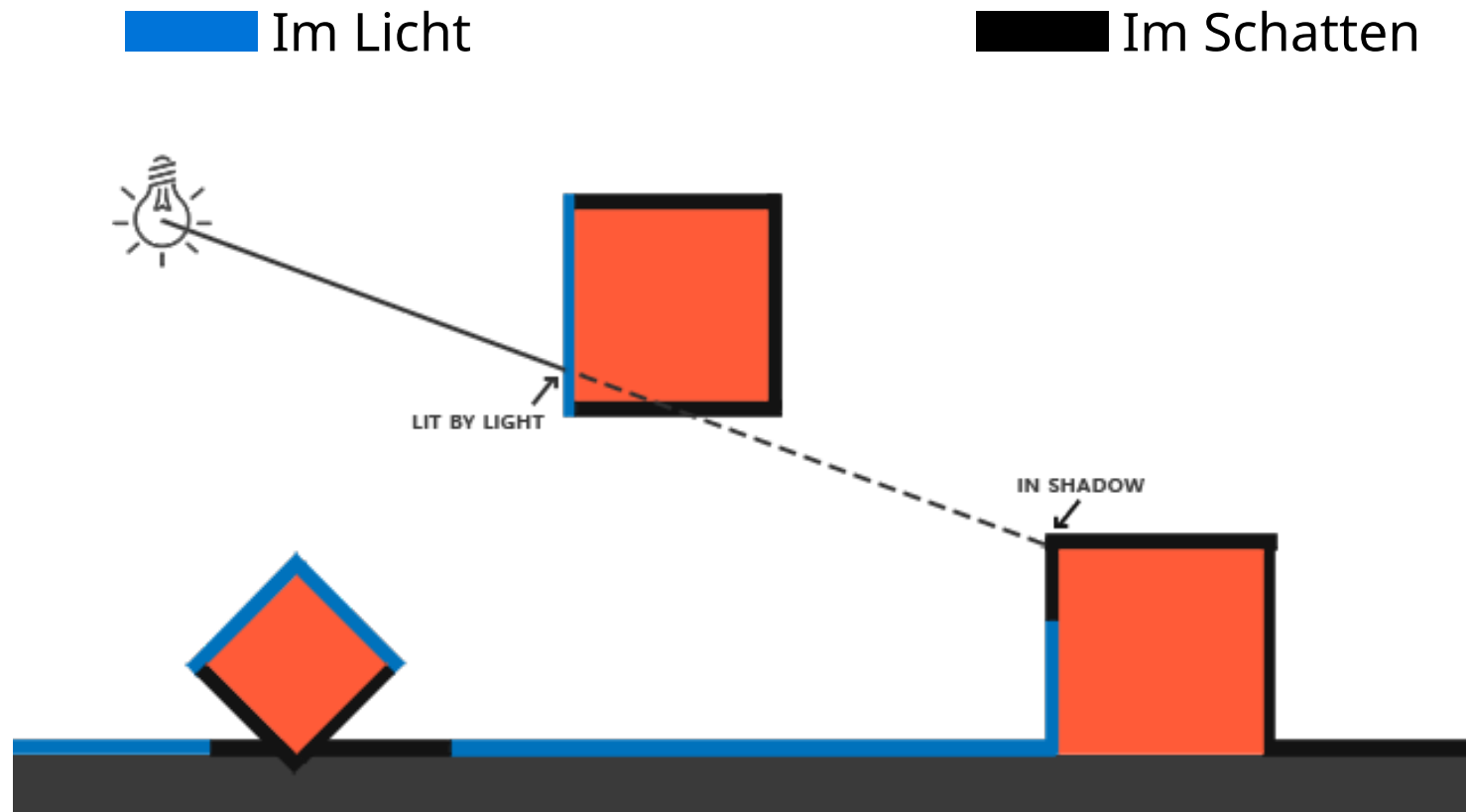
- Aliasing

Nutzung

- Blender EEVEE, Unreal, Unity ...

Ansatz

- Szene wird von der Lichtquelle aus gerendert
- Alles was von der Lichtquelle aus gesehen werden kann ist beleuchtet
- Alles was nicht von der Lichtquelle gesehen werden kann ist im Schatten



Ansatz

- Erster Hit des Lichttrays wird mit allen folgenden Hits verglichen
- Wenn der neue Punkt weiter entlang des Rays patziert ist, ist der im Schatten
- Rays zu ineffizient für real time Rendering
- Depth Buffer als bessere Alternative

Shadow Map

1. Pass

Rendern der Szene aus Sicht der Lichtquelle

- Tiefenwerte pro Pixel abspeichern im z-Buffer
- Die Geometrie der Szene wird einmal pro Punktlicht gerendert
- Für ein omnidirektionales Punktlicht muss die Szene mehr als einmal gerendert werden
- Sofern der Schattenbereich nicht auf einen gewünschten Wirkungswinkel für gerichtetes Licht (z.B.: den der Sonne) begrenzt werden kann, sollte eine Parallelprojektion verwendet werden

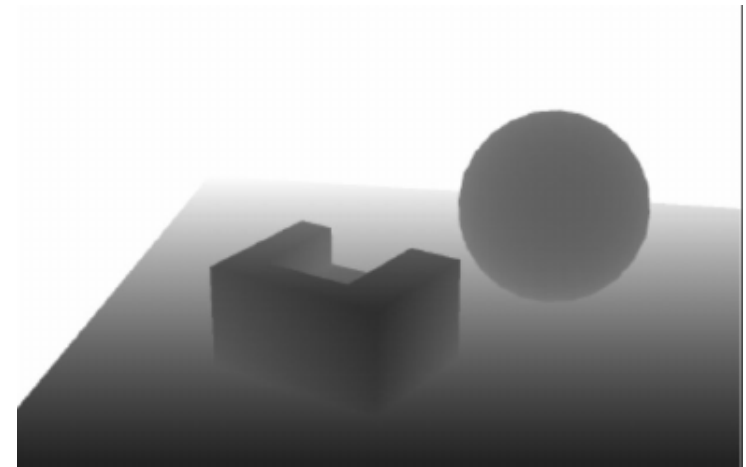


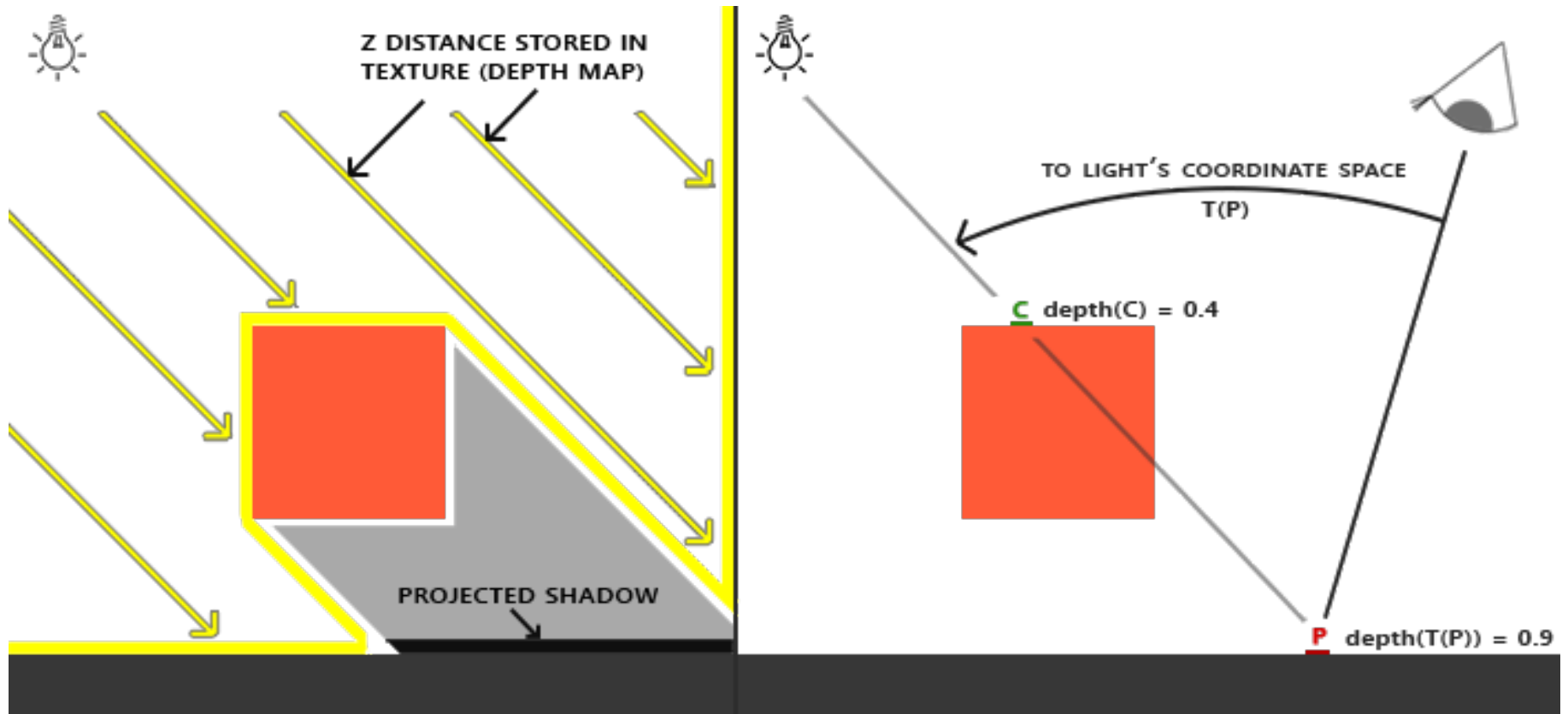
Figure 7: Szene, die aus der Sicht der Lichtquelle gerendert wurde. Weiß ist fern und schwarz ist nah.

Shadow Map

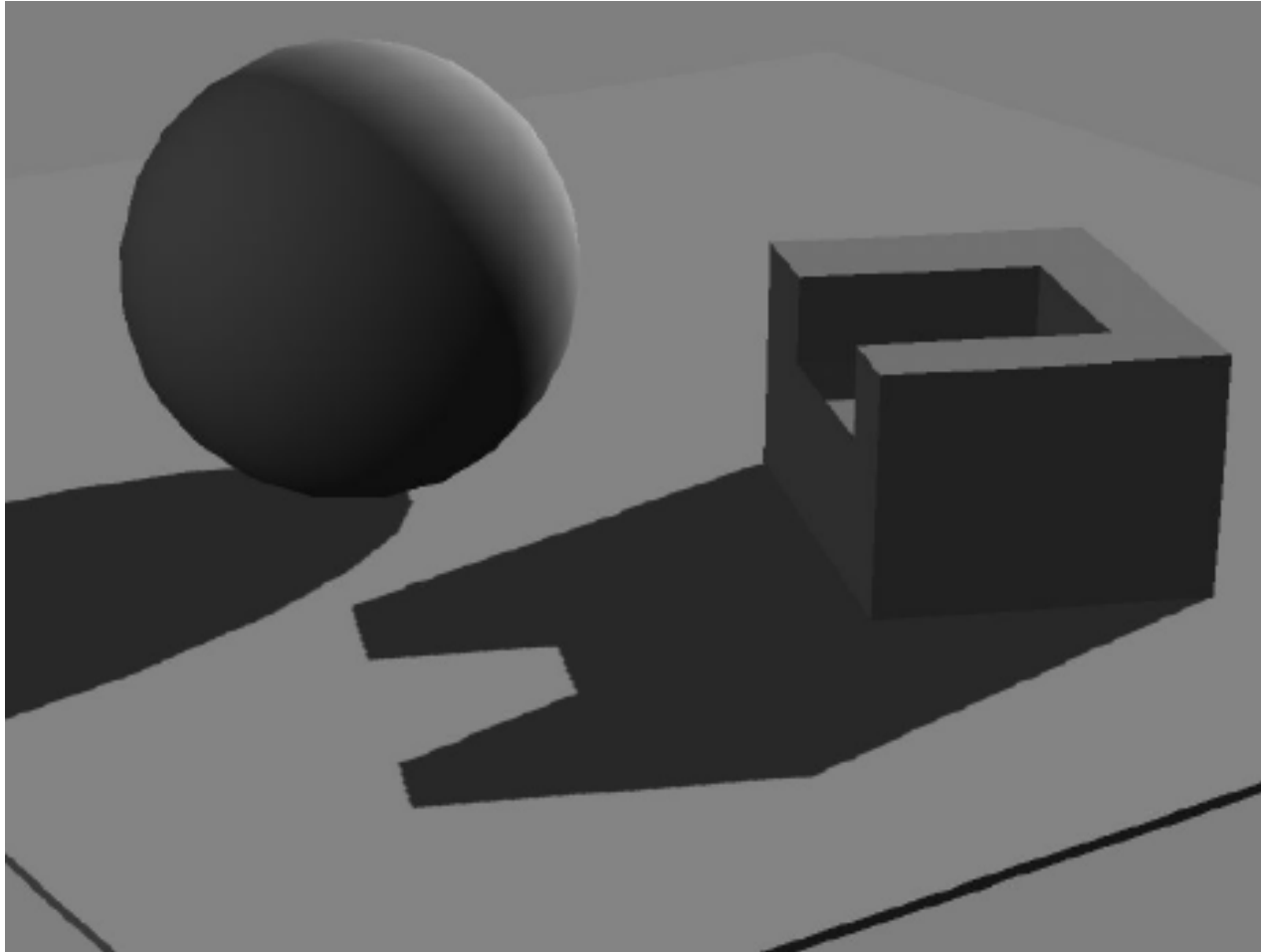
2. Pass

- Render aus Sicht der Kamera
- Verwendung der Shadow Map
 - Umwandlung des Oberflächenpunkts P in Lichtkoordinaten P_L
 - Vergleich der aktuellen Oberflächentiefe am Punkt P mit der gespeicherten Tiefe in der Shadow Map
 - Wenn $T > T_{\text{gespeichert}} \rightarrow$ Pixel ist im Schatten
 - Wenn Tiefe $T == T_{\text{gespeichert}}$ gespeicherte Tiefe denn ist der Pixel beleuchtet
 - $T < T_{\text{gespeichert}}$ sollte nie eintreten

Shadow Map



Ergebnis



Multiple Lights

- Eine Shadow Map pro Lichtquelle
- Punktlicht: bis zu 6 Shadow Maps
- Definierung von Schattenerzeugern und Empfängern

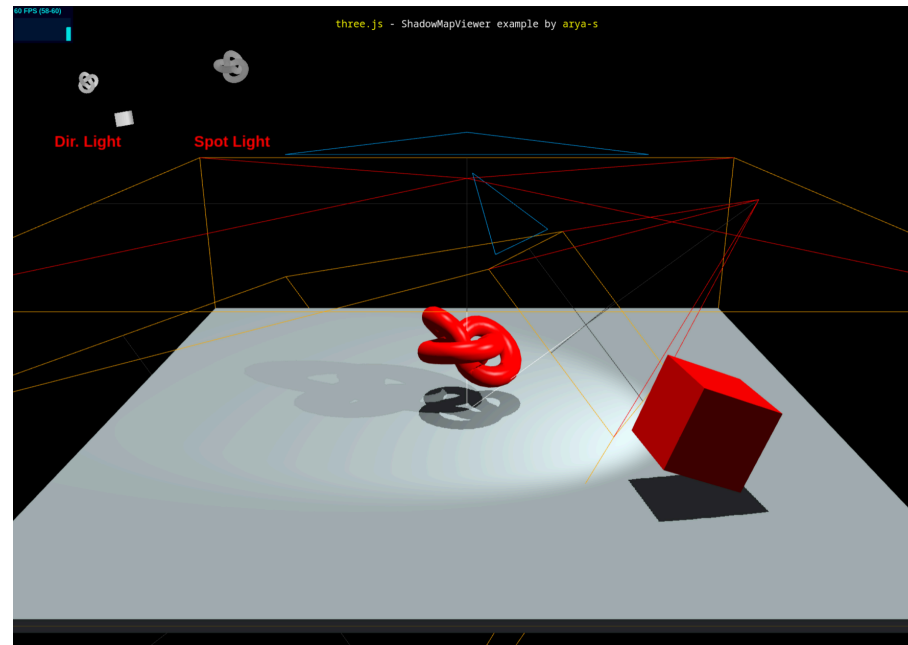


Figure 10: Zwei Shadow Maps in einem ThreeJS Beispiel.

[Link](#) →

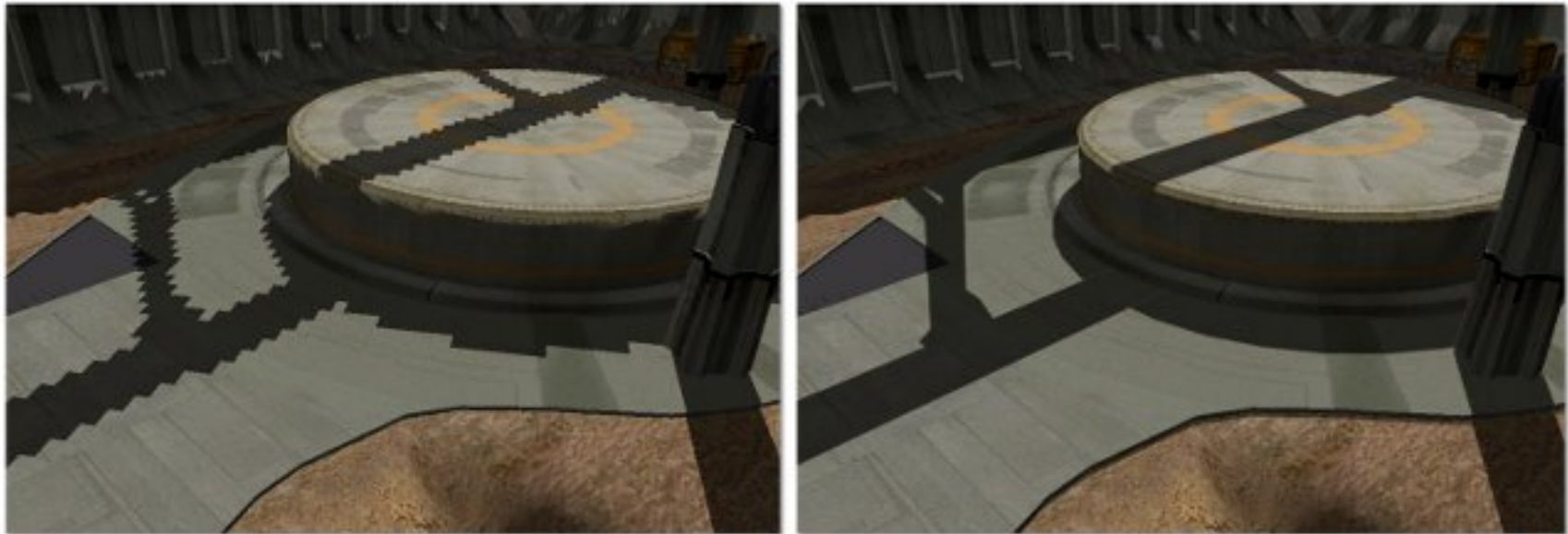
Multiple Lights

```
1  const spotLight = new THREE.SpotLight(...);  
2  const directionalLight = new THREE.DirectionalLight(...);  
3  torusKnot.castShadow = true;  
4  torusKnot.receiveShadow = true;  
5  cube.castShadow = true;  
6  cube.receiveShadow = true;  
7  ground.castShadow = false;  
8  ground.receiveShadow = true;
```

JS JavaScript

Aliasing Probleme

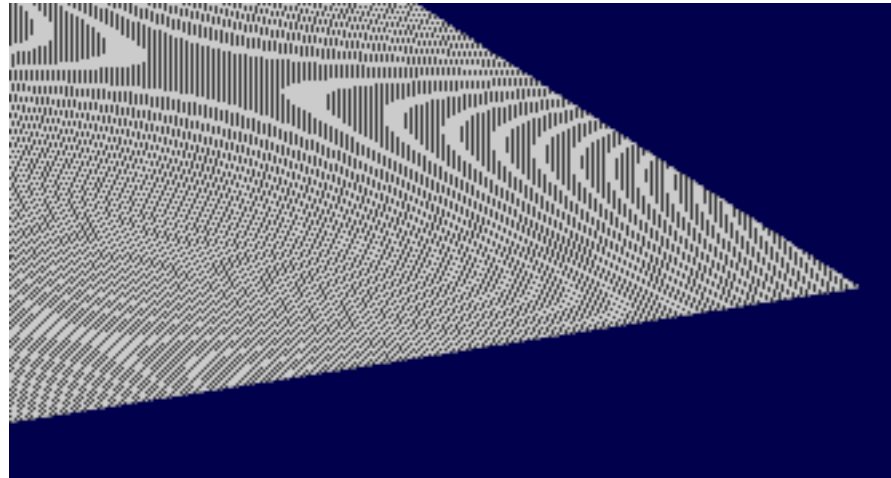
- Vergrößerung, Verkleinerung und anisotrope Artefakte
- Erhöhung der Auflösung der Shadow Map ist rechenintensiv
- Texturfilterung kann nicht auf standard Shadow Maps angewendet werden



Schattenakne

$$T_{\text{gepeichert}} > T$$

- Numerische Fehler
- Führen zu Artefakten in Bereichen die beleuchtet sein sollten



Anti Aliasing

Es gibt Varianten des Shadow Mapping, die Anti-Aliasing Techniken unterstützen. Normalerweise werden sie bei Texturfilterung angewendet

Techniken: bi- und trilineare Interpolation, Mip-Mapping und Weichzeichnen

- Variance Shadow Maps
- Exponential Shadow Maps
- Cascaded Shadow Maps

Cascaded Shadow Maps

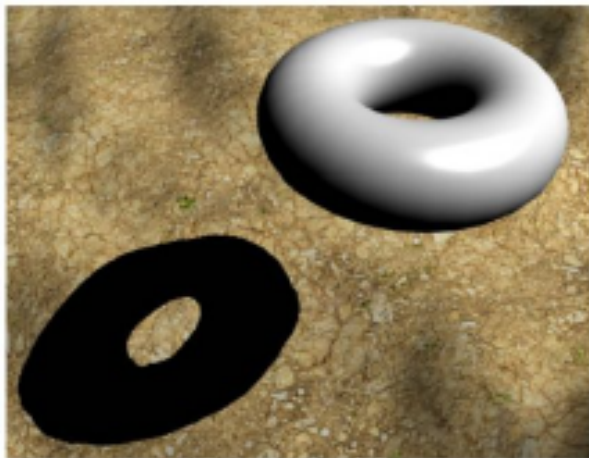
- A: keine Cascades
- B: 2 Cascades
- C: 3 Cascades
- D: 4 Cascades



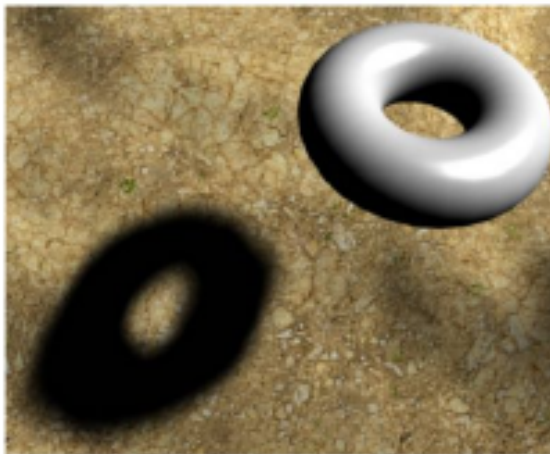
Konzept: Shadow Maps werden hochauflösender, wenn sie näher am Betrachter sind

Soft Shadows

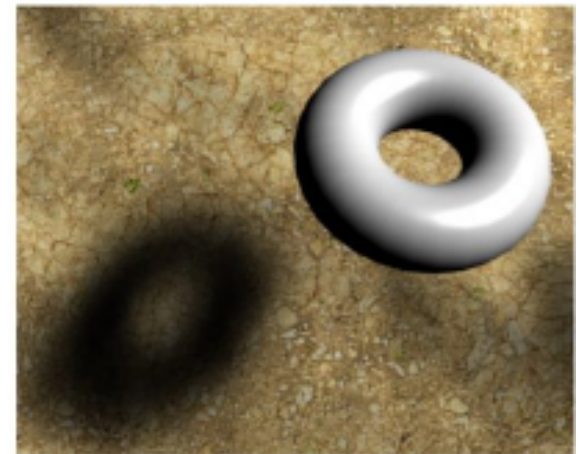
- Verwischen der Shadow Map
- Erzeugt Eindruck weicher Schatten



no blurring



some blurring

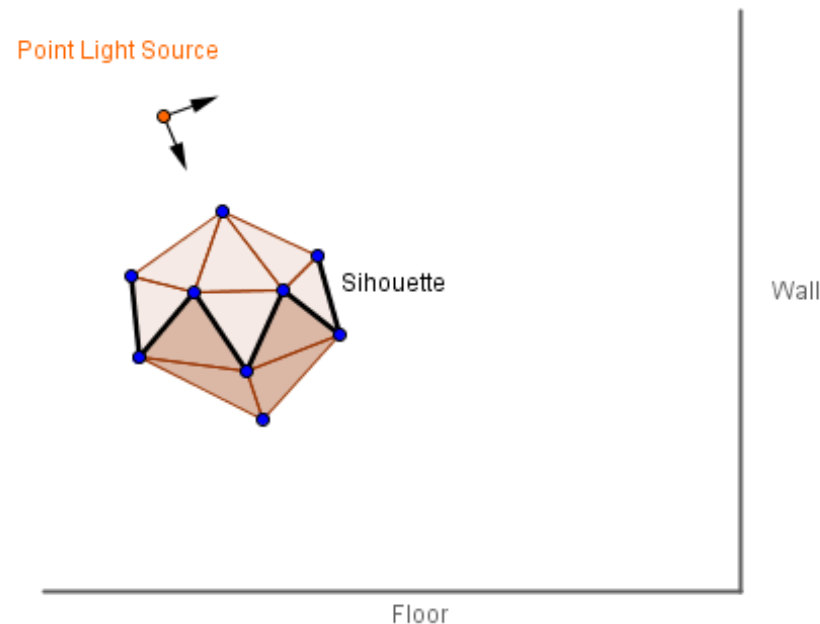


more blurring

Shadow Volumes

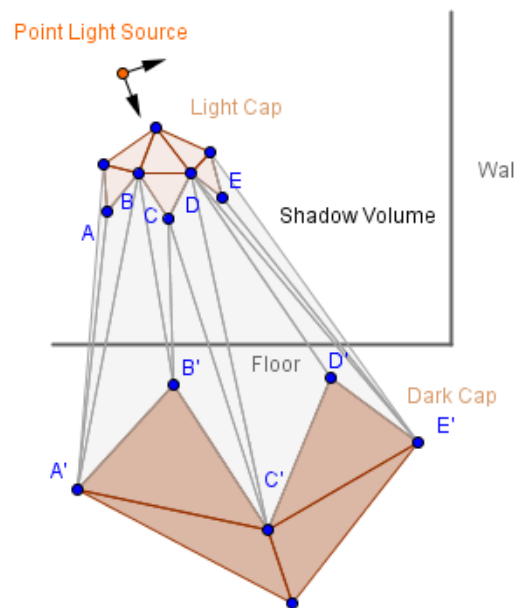
Shadow Volumes & Stencil Shadows

- Popularisiert von Doom 3 (2004)
 - Hauptidee: Erstelle ein Volumen, welches alle Schatten der Szene einschließt
 - Finden der Silhouetten aller Objekte
- Kanten finden, bei denen ein Face Richtung Licht zeigt und eins weg vom Licht zeigt



Shadow Volumes & Stencil Shadows

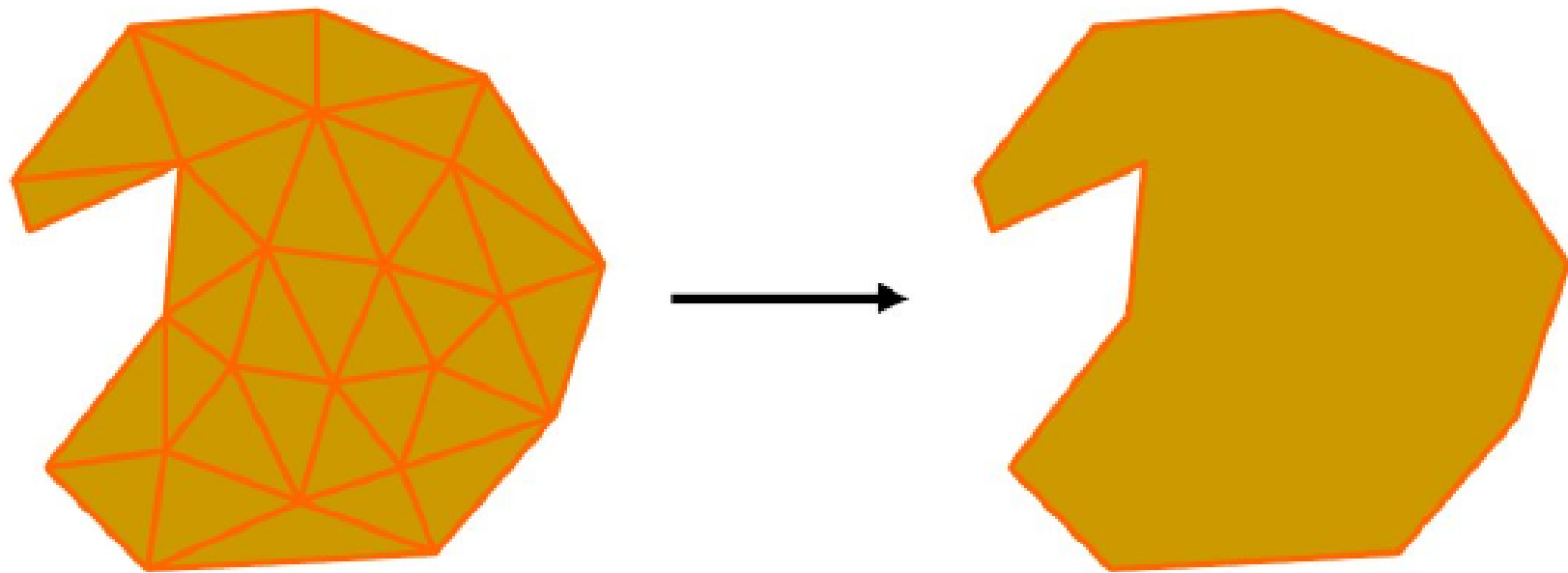
- Vertices, die Richtung Licht zeigen bleiben an ihrer Position
- Vertices, die weg vom Licht zeigen werden weit weg extrudiert
- Duplizieren und Extrudieren der Silhouetten-Vertices und Verbindung mit den originalen Vertices



Erstellung von: Light Cap, Dark Cap und Sides

Silhouette Edges

- Nur äußere Kanten tragen zur Erstellung des Shadow Volumes bei
- Erstellung der Silhouettenkante entfernt nutzlose Polygone



Shadow Volumes & Stencil Shadows

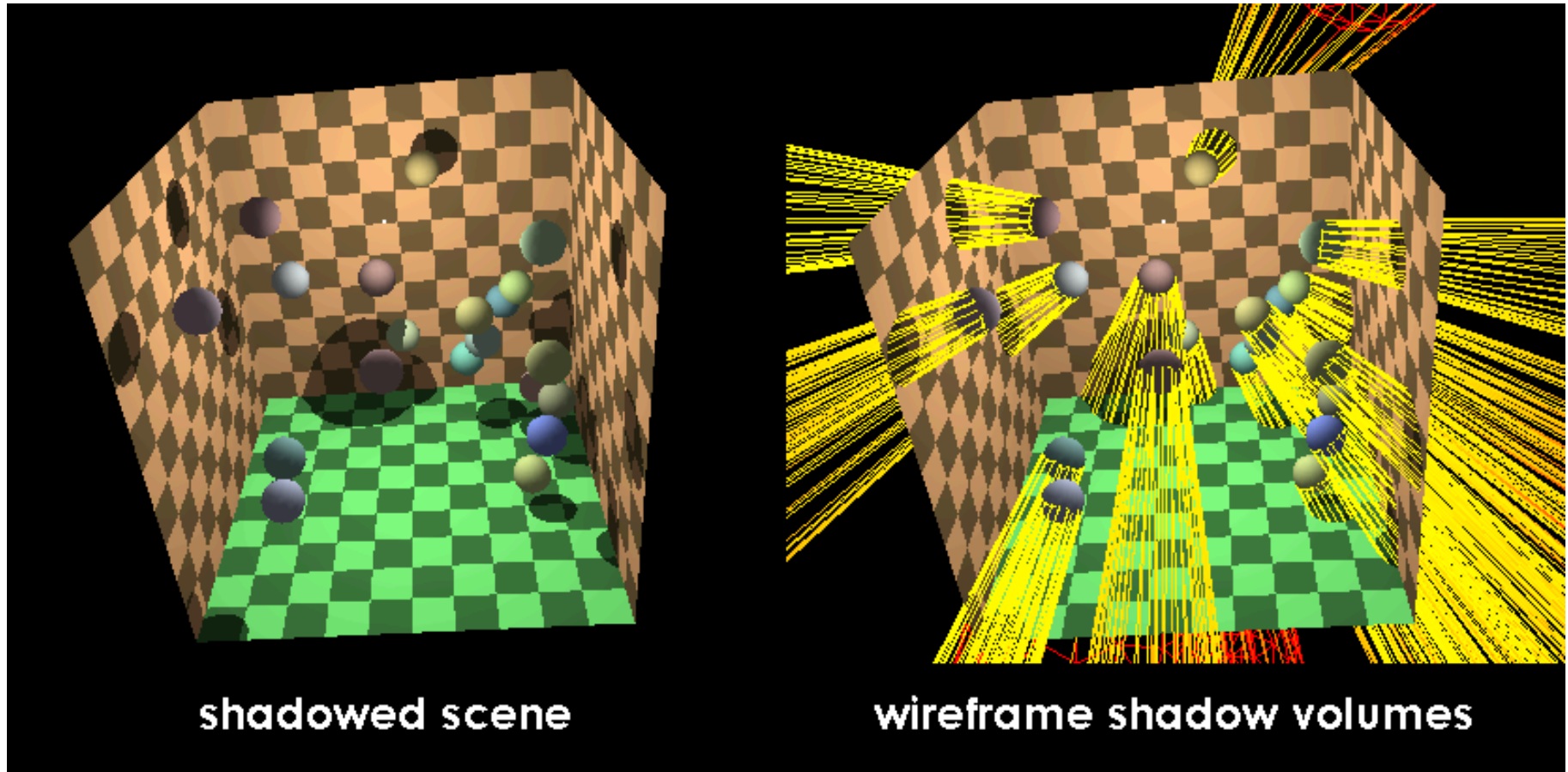
Pros

- Hohe Qualität
- Scharfe Kanten

Cons

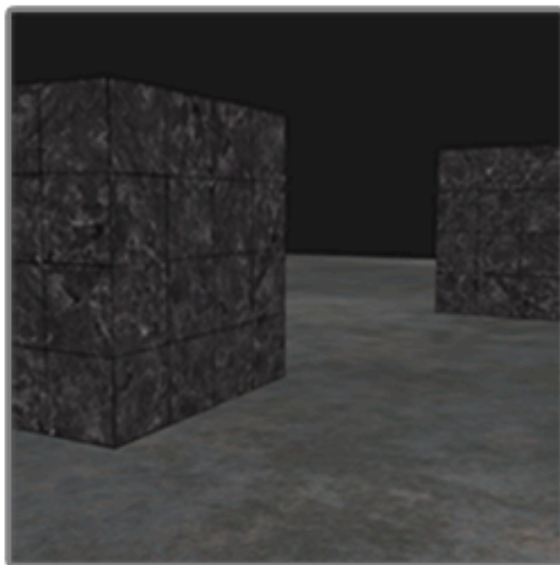
- Nicht so schnell wie Shadow Maps
- Sensitiver gegenüber geometrischer Komplexität
- Kanten teilweise zu scharf

Shadow Volumes



Stencil Buffer

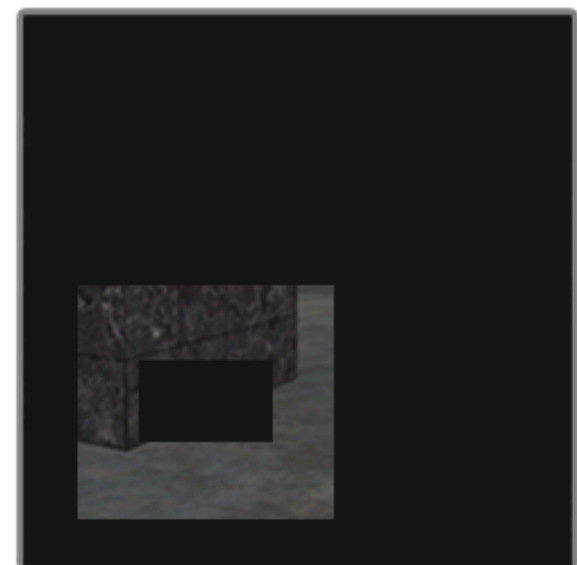
- Buffer mit 8-Bit pro Pixel
- Hinzufügen und Entfernen von Inhalten beim Rendering
- Maskierung im finalen Bild



Color buffer



Stencil buffer



After stencil test

Shadow Volumes & Stencil Buffer

Implementierung

1. Rendering der Szene mit ambienten Licht
2. Rendering der Front Facing Shadow Volumes in den Stencil Buffer
3. Rendering der Back Facing Shadow Volumes in den Stencil Buffer
4. Rendering diffuser und specular Oberflächen, bei denen der Buffer 0 ist

Beispiel



Figure 18: Carmacks Stencil Buffer, Doom 3

Reflexionen

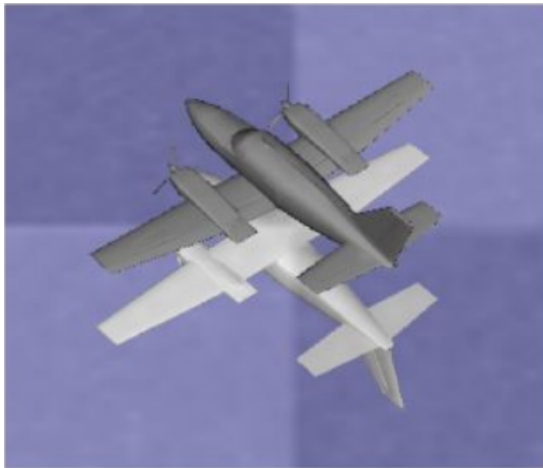
Planare Reflexionen

Reflexionen durch Environment Mapping

- Simulation von Reflexionen durch Environment Mapping auf gekrümmten Oberflächen
- Funktioniert nicht bei ebenen Oberflächen
- Weitere Einschränkungen: Reflektierte Objekte wirken weiter weg

Planare Reflexionen

- Geben Hinweise auf räumliche Beziehung und erhöhen Realismus



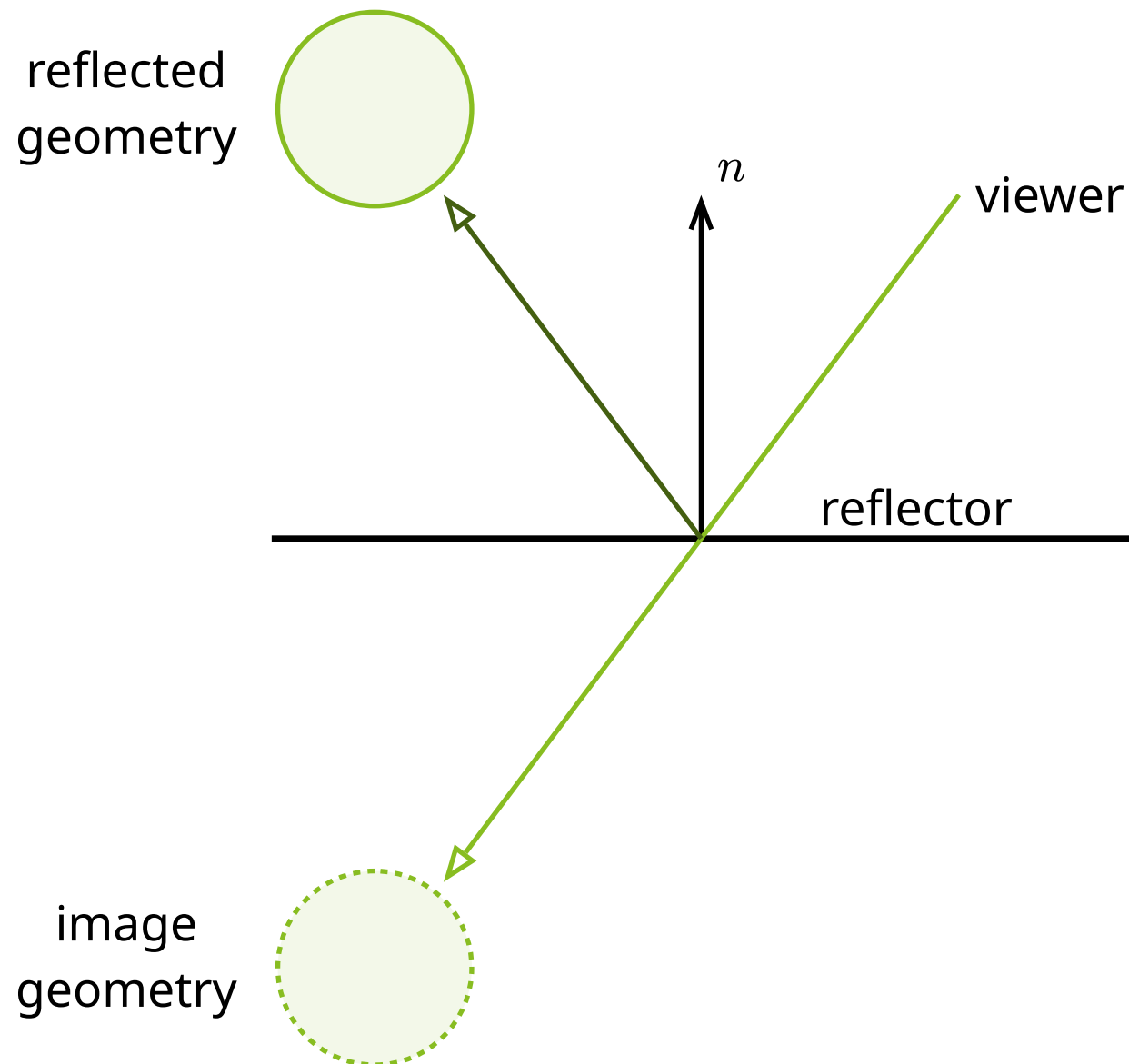
Planare Reflexionen

Grundlegende Idee

- Kopie des zu reflektierenden Objektes erstellen
- Kopie in die reflektierte Position transformieren und rendern

Gesetz der Reflexion Eingehender Winkel = Ausgehender Winkel

Planare Reflexion



Planare Reflexion

- Annahme: Ebene ist $z = 0$
- Es wird um die z -Achse gespiegelt
- Beispiel: `glScalef(x: 1, y: 1, z: -1);`
- Backfacing wird zu Frontfacing
- Lichtquellen sollten auch reflektiert werden
- Clipping mit Stencil Buffer benötigt

Planare Reflexionen

Schritte des Renderns:

1. Reflektierende Teile der Grundplatte in den Stencil Buffer
2. Das skalierte Modell $((1, 1, -1))$ wird mit Stencil Buffer maskiert
3. Rendern der Grundplatte (halbtransparent)
4. Rendern des nicht skalierten Modells

